

Names: Asifa Iqbal, Jocelynn Lopez, Alexa Lunn, Daniel Parent

AI Sources Used

- Gemini
- Claude
- ChatGPT

AI Problem Identification & Proposal

The Ferrari SF-25 race car has been consistently underperforming in the 2025 Formula 1 season, demonstrating a significant lack of stability and a narrow operating window. The car is highly sensitive to track conditions, ride height, and small setup changes, resulting in inconsistent performance between qualifying and race day and across different circuit types. If Ferrari is able to pinpoint exactly what is causing these inconsistencies and resolve them to optimize the car for success on each circuit type, they can perform better when it matters, which benefits everyone on the team at all levels.

Everyone on the engineering team at Ferrari will benefit from clear, data-driven recommendations for design changes based on what is causing inconsistent performance. The strategy team will also gain predictive models for speed and tire degradation under different track conditions, leading to better pit stop and tire strategy calls. Finally, the drivers will benefit from having a more predictable, stable car that can be pushed to its limits, reducing driver error and maximizing race potential for the changing circumstances of each race and track.

The data that we will be using is Driver data, Race data, Track data, and Race Result data. Most relevant to the problem will be attributes measuring the condition and height of each track, the difference between the finishing time and speed from the leader, and the tire degradation percentage.

The system will support car performance prediction models, performance trend analysis, race prediction models, and models supporting identification of car defects.

AI Ready Database Design

Driver	ID (PK) First Name Last Name Date Hired Role
Race	ID (PK) Name Date

	Track ID (FK) Date Started
Track	ID (PK) Name Location Opening Date Capacity Smoothness Index
Race Result	ID (PK) Driver ID (FK) Race ID (FK) Track ID (FK) Finished? (Y/N) Finishing Place Quali Delta to Pole Sec Race Pace Delta to Leader Sec Ride Height Dev mm Tire Deg Rate Percent
Driver Track Experience	Driver Track Experience_ID (PK) Driver ID Track ID Experience Level
Bumpy Track Penalty	Bumpy Track Penalty ID (PK) Race ID Track ID Penalty Points
Sensitivity Hotspot	Sensitivity Hotspot ID (PK) Race Result ID Description
Weather	Weather ID (PK) Race ID Good Weather Conditions Bad Weather Conditions

Entities:

- Driver
- Track
- Race
- Race Result
- Driver Track Experience
- Bumpy Track Penalty
- Sensitivity Hotspot
- Weather

Relationships:

- A **driver** can have **zero or many** race results; each result is unique to one driver
- **Race** can have **zero** (if in the future) **or many** race results; each result is unique to one race
- A **track** can have **zero or many** race results; each result is unique to one track
- A **race** can only have **one** track; a **track** can be used for **zero or many** races

DriverTrackExperience Relationships

- A **driver** can have **zero or many** driver–track experience records.
- A **track** can have **zero or many** driver–track experience records.
- Each **DriverTrackExperience** entry must refer to **one driver** and **one track**.

BumpyTrackPenalty Relationships

- A **race** can have **zero or many** bumpy-track penalty evaluations.
- A **track** can have **zero or many** bumpy-track penalty evaluations.
- Each **BumpyTrackPenalty** entry is associated with **one race** and **one track**.

SensitivityHotspot Relationships

- A **race result** can have **zero or many** sensitivity hotspot records.
- Each **SensitivityHotspot** entry is associated with **one race result**.

Weather Relationships

- A **race** can have **zero or one** associated weather record (optional).
- Each **weather** record is associated with **one race only**.

Business Rules:

Driver & Track

- New drivers and tracks can be recorded without race results.
- Each driver must have one role; a role may apply to many drivers.
- Drivers can have zero or many race results and driver–track experience records.
- Tracks can host zero or many races and can have zero or many race results and driver–track experience records.

Race & Race Result

- Each race occurs on one track; a track can host many races.

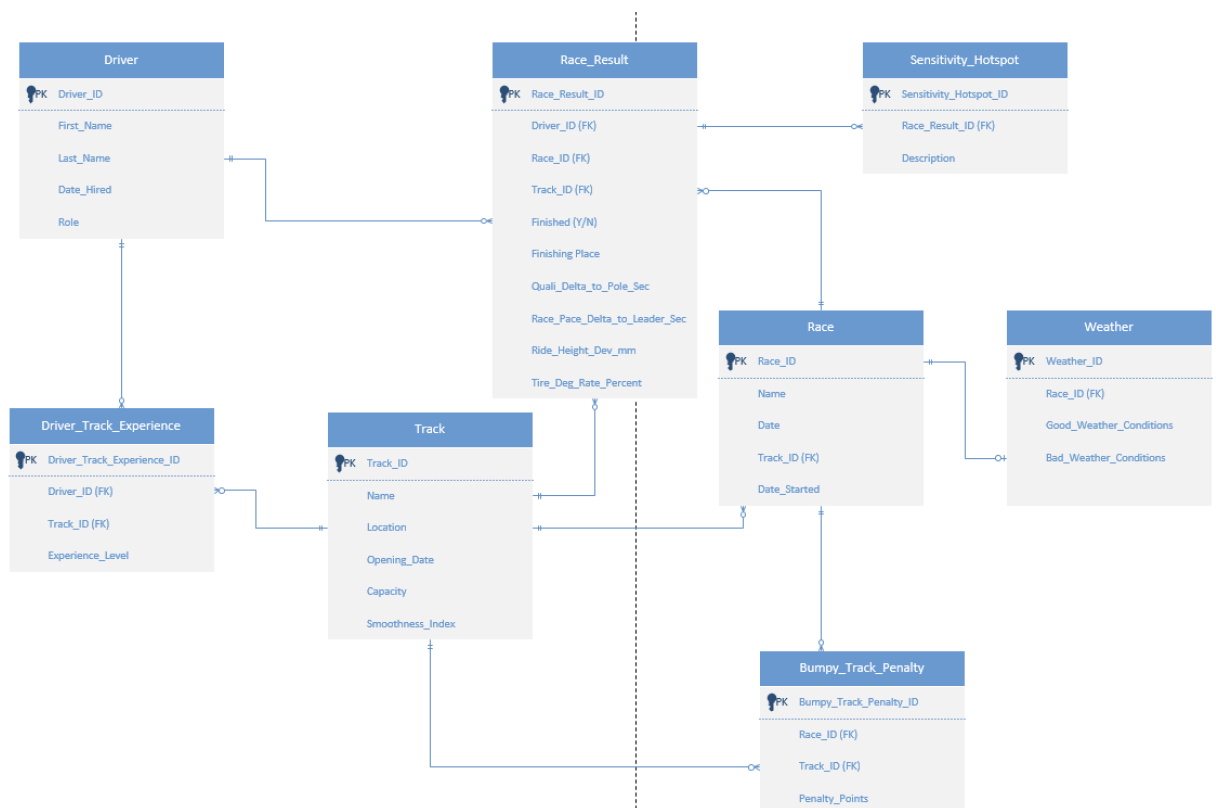
- Races can have zero or many race results; each result is linked to one driver, one race, and one track.
- Race results can have zero or many sensitivity hotspot records.
- Future and past races can both be recorded.

Track Condition & Weather

- BumpyTrackPenalty entries link one race and one track; races and tracks can have many penalty records.
- Each race can have zero or one weather record.

Performance & Data Integrity

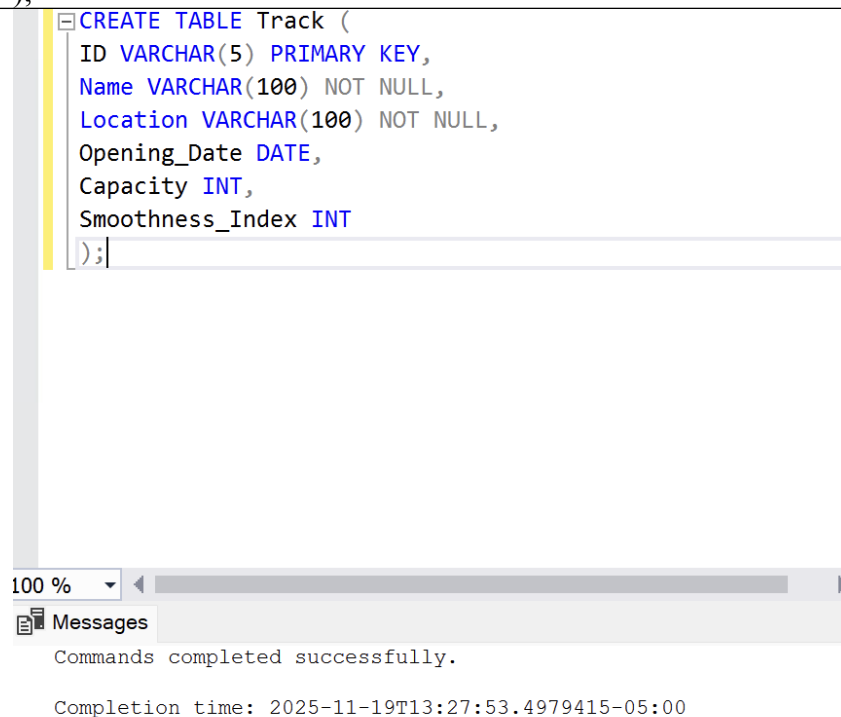
- Ride height deviation, tire degradation, and performance deltas are stored per race result.
- Data supports performance trend analysis, defect identification, and predictive modeling.
- All IDs are unique; foreign keys must reference existing records; optional fields may be null.



Logical & Physical Design

SQL Syntax: (creating track table)

```
CREATE TABLE Track (  
ID VARCHAR(5) PRIMARY KEY,  
Name VARCHAR(100) NOT NULL,  
Location VARCHAR(100) NOT NULL,  
Opening_Date DATE,  
Capacity INT,  
Smoothness_Index INT  
);
```



SQL Syntax: (creating driver table)

```
CREATE TABLE Driver (  
ID VARCHAR(5) PRIMARY KEY,  
First_Name VARCHAR(50) NOT NULL,  
Last_Name VARCHAR(50) NOT NULL,  
Date_Hired DATE,  
Role VARCHAR(50)  
);
```

Screenshot of Query and Results in SQL Server:

```
CREATE TABLE Driver (  
  ID VARCHAR(5) PRIMARY KEY,  
  First_Name VARCHAR(50) NOT NULL,  
  Last_Name VARCHAR(50) NOT NULL,  
  Date_Hired DATE,  
  Role VARCHAR(50)  
);
```

100 %



Messages

Commands completed successfully.

Completion time: 2025-11-19T13:27:53.4979415-05:00

SQL Syntax: (creating race table)

```
CREATE TABLE Race (  
  ID VARCHAR(5) PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Date DATE NOT NULL,  
  Track_ID VARCHAR(5) NOT NULL,  
  Date_Started DATE,  
  FOREIGN KEY (Track_ID) REFERENCES Track(ID)  
);
```

Screenshot of Query and Results in SQL Server:

```
CREATE TABLE Race (  
  ID VARCHAR(5) PRIMARY KEY,  
  Name VARCHAR(100) NOT NULL,  
  Date DATE NOT NULL,  
  Track_ID VARCHAR(5) NOT NULL,  
  Date_Started DATE,  
  FOREIGN KEY (Track_ID) REFERENCES Track(ID)  
);
```

00 %

Messages

Commands completed successfully.

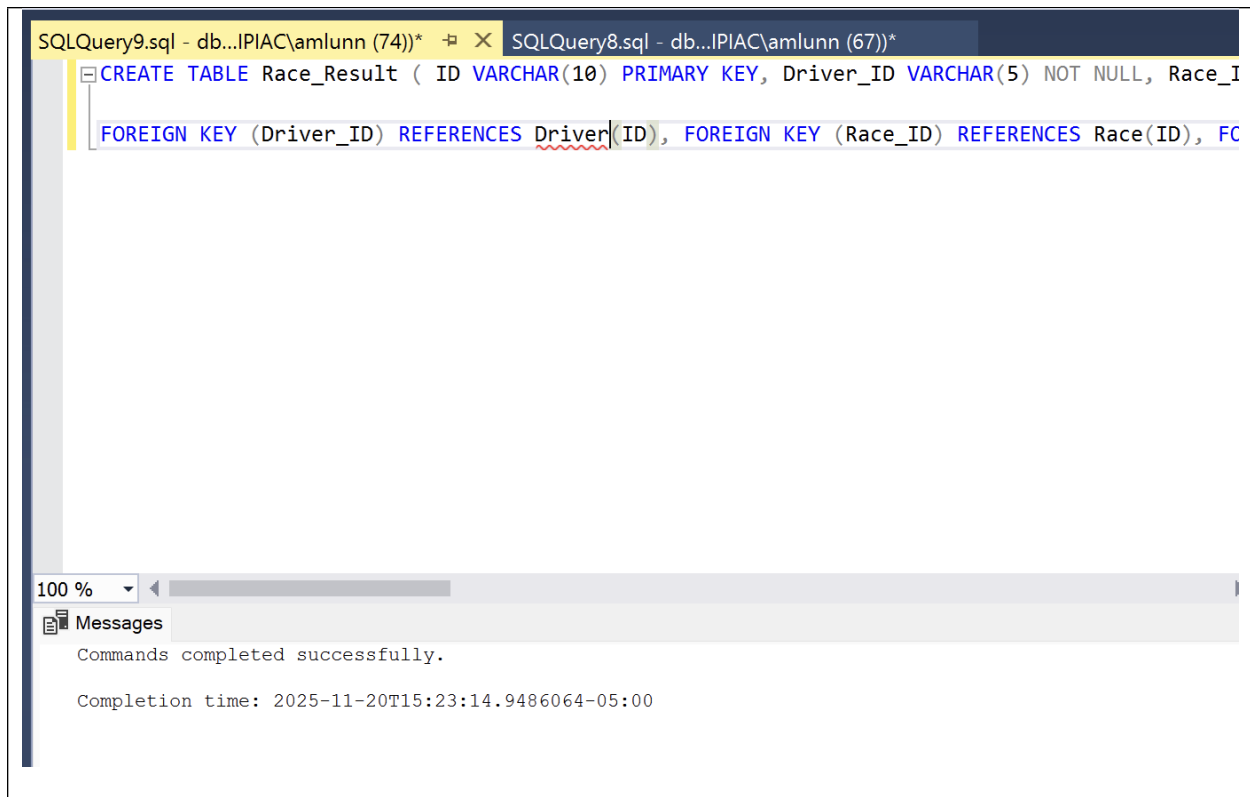
Completion time: 2025-11-19T14:01:47.7653708-05:00

SQL Syntax: (creating race_result table)

```
CREATE TABLE Race_Result ( ID VARCHAR(10) PRIMARY KEY, Driver_ID  
  VARCHAR(5) NOT NULL, Race_ID VARCHAR(5) NOT NULL, Track_ID VARCHAR(5)  
  NOT NULL, Finished CHAR(1) NOT NULL, Finishing_Place DECIMAL(10),  
  Quali_Delta_to_Pole_Sec DECIMAL(4,3), Race_Pace_Delta_to_Leader_Sec  
  DECIMAL(3,1), Ride_Height_Dev_mm DECIMAL (3,1), Tire_Deg_Rate_Percent  
  DECIMAL (3,2),
```

```
FOREIGN KEY (Driver_ID) REFERENCES Driver(ID), FOREIGN KEY (Race_ID)  
REFERENCES Race(ID), FOREIGN KEY (Track_ID) REFERENCES Track(ID) );
```

Screenshot of Query and Results in SQL Server:



SQL Implementation & AI Future Engineering

Core SQL:

NOTE: All screenshots for this section (inserting data) were taken prior to adding more data to each table later on for greater analysis. The process was the same for each table.

-- Inserting data into Track

INSERT INTO Track (ID, Name, Location, Opening_Date, Capacity, Smoothness_Index) VALUES

('T001', 'Monza', 'Monza, Italy', '1922-09-03', 115000, 4),

('T002', 'Circuit of the Americas (COTA)', 'Austin, USA', '2012-11-18', 150000, 1),

('T003', 'Marina Bay Street Circuit', 'Singapore', '2008-09-28', 87000, 3),

('T004', 'Silverstone Circuit', 'Silverstone, UK', '1948-10-02', 142000, 5)

('T005', 'Suzuka', 'Suzuka, Japan', '1962-11-03', 155000, 4),

('T006', 'Interlagos', 'São Paulo, Brazil', '1940-05-12', 70000, 2),
('T007', 'Bahrain International Circuit', 'Sakhir, Bahrain', '2004-03-17', 70000, 5),
('T008', 'Barcelona-Catalunya', 'Barcelona, Spain', '1991-09-29', 140700, 3),
('T009', 'Imola', 'Imola, Italy', '1953-04-25', 95000, 4),
('T010', 'Zandvoort', 'Netherlands', '1948-08-07', 105000, 2),
('T011', 'Hungaroring', 'Budapest, Hungary', '1986-08-10', 70000, 3),
('T012', 'Red Bull Ring', 'Spielberg, Austria', '1970-08-16', 105000, 5),
('T013', 'Jeddah Corniche Circuit', 'Jeddah, Saudi Arabia', '2021-12-05', 110000, 2),
('T014', 'Losail International Circuit', 'Qatar', '2004-10-02', 80000, 3),
('T015', 'Shanghai International Circuit', 'Shanghai, China', '2004-09-26', 200000, 4),
('T016', 'Albert Park', 'Melbourne, Australia', '1996-03-10', 140000, 2),
('T017', 'Montreal Circuit Gilles Villeneuve', 'Montreal, Canada', '1978-10-08', 100000, 3),
('T018', 'Mexico City Autodromo Hermanos Rodriguez', 'Mexico City, Mexico', '1962-11-04', 110000, 2),
('T019', 'Las Vegas Strip Circuit', 'Las Vegas, USA', '2023-11-18', 120000, 1),
('T020', 'Spa-Francorchamps', 'Spa, Belgium', '1925-08-18', 120000, 4);

```
SQLQuery1.sql - db...IPIAC\aiqbal3 (61))* X
-- Inserting data into Track
INSERT INTO Track (ID, Name, Location, Opening_Date, Capacity, Smoothness_Index) VALUES
('T001', 'Monza', 'Monza, Italy', '1922-09-03', 115000, 4),
('T002', 'Circuit of the Americas (COTA)', 'Austin, USA', '2012-11-18', 150000, 1),
('T003', 'Marina Bay Street Circuit', 'Singapore', '2008-09-28', 87000, 3),
('T004', 'Silverstone Circuit', 'Silverstone, UK', '1948-10-02', 142000, 5);

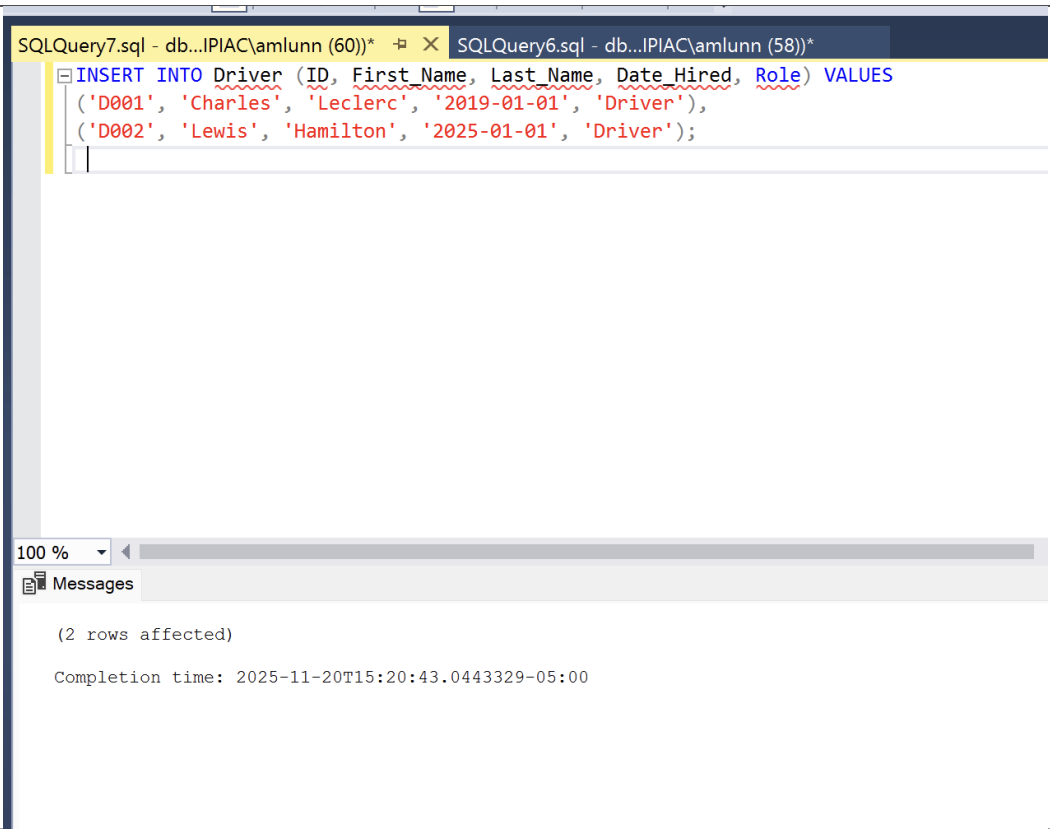
77 %
Messages
(4 rows affected)
Completion time: 2025-11-20T14:13:22.4130092-05:00
```

-- Inserting data into Driver

```
INSERT INTO Driver (ID, First_Name, Last_Name, Date_Hired, Role)
VALUES
```

```
('D001', 'Charles', 'Leclerc', '2019-01-01', 'Driver'),
('D002', 'Lewis', 'Hamilton', '2025-01-01', 'Driver')
('D003', 'Oliver', 'Bearman', '2024-01-01', 'Reserve Driver'),
('D004', 'Robert', 'Shwartzman', '2022-01-01', 'Test Driver'),
('D005', 'Arthur', 'Leclerc', '2023-01-01', 'Academy Driver'),
('D006', 'Dino', 'Beganovic', '2023-01-01', 'Academy Driver'),
('D007', 'James', 'Wharton', '2023-01-01', 'Academy Driver'),
('D008', 'Rafael', 'Camara', '2024-01-01', 'Academy Driver'),
('D009', 'Jock', 'Clear', '2025-01-01', 'Simulator Driver'),
('D010', 'Luca', 'Baldisserrì', '2025-01-01', 'Simulator Driver'),
('D011', 'Giuliano', 'Alesi', '2024-01-01', 'Academy Driver'),
('D012', 'Callum', 'Ilott', '2021-01-01', 'Test Driver'),
```

('D013', 'Mick', 'Schumacher', '2021-01-01', 'Reserve Driver'),
('D014', 'Antonio', 'Giovinazzi', '2022-01-01', 'Test Driver'),
('D015', 'Marcus', 'Armstrong', '2023-01-01', 'Academy Driver'),
('D016', 'Enzo', 'Fittipaldi', '2024-01-01', 'Academy Driver'),
('D017', 'Jak', 'Crawford', '2025-01-01', 'Academy Driver'),
('D018', 'Sebastian', 'Montoya', '2025-01-01', 'Academy Driver'),
('D019', 'Bear', 'Smith', '2025-01-01', 'Simulator Driver'),
('D020', 'Lorenzo', 'Fluxa', '2024-01-01', 'Academy Driver');



Race

INSERT INTO Race (ID, Name, Date, Track_ID, Date_Started) VALUES
('R001', 'British Grand Prix', '2025-07-06', 'T004', '2025-07-04'),

('R002', 'Singapore Grand Prix', '2025-09-21', 'T003', '2025-09-19'),
('R003', 'United States Grand Prix', '2025-10-19', 'T002', '2025-10-17'),
('R004', 'Italian Grand Prix', '2025-09-07', 'T001', '2025-09-05')
('R005', 'Japanese Grand Prix', '2025-04-13', 'T005', '2025-04-11'),
('R006', 'Brazilian Grand Prix', '2025-11-09', 'T006', '2025-11-07'),
('R007', 'Bahrain Grand Prix', '2025-03-09', 'T007', '2025-03-07'),
('R008', 'Spanish Grand Prix', '2025-06-01', 'T008', '2025-05-30'),
('R009', 'Emilia Romagna Grand Prix', '2025-05-18', 'T009', '2025-05-16'),
('R010', 'Dutch Grand Prix', '2025-08-24', 'T010', '2025-08-22'),
('R011', 'Hungarian Grand Prix', '2025-07-20', 'T011', '2025-07-18'),
('R012', 'Austrian Grand Prix', '2025-06-29', 'T012', '2025-06-27'),
('R013', 'Saudi Arabian Grand Prix', '2025-03-23', 'T013', '2025-03-21'),
('R014', 'Qatar Grand Prix', '2025-11-30', 'T014', '2025-11-28'),
('R015', 'Chinese Grand Prix', '2025-04-27', 'T015', '2025-04-25'),
('R016', 'Australian Grand Prix', '2025-03-16', 'T016', '2025-03-14'),
('R017', 'Canadian Grand Prix', '2025-06-15', 'T017', '2025-06-13'),
('R018', 'Mexico City Grand Prix', '2025-10-26', 'T018', '2025-10-24'),
('R019', 'Las Vegas Grand Prix', '2025-11-22', 'T019', '2025-11-20'),
('R020', 'Belgian Grand Prix', '2025-08-03', 'T020', '2025-08-01');

SQLQuery4.sql - db...IPIAC\amlunn (83))* SQLQuery3.sql - db...IPIAC\amlunn (62))*

```
INSERT INTO Race (ID, Name, Date, Track_ID, Date_Started) VALUES
('R001', 'British Grand Prix', '2025-07-06', 'T004', '2025-07-04'),
('R002', 'Singapore Grand Prix', '2025-09-21', 'T003', '2025-09-19'),
('R003', 'United States Grand Prix', '2025-10-19', 'T002', '2025-10-17'),
('R004', 'Italian Grand Prix', '2025-09-07', 'T001', '2025-09-05');
```

100 %

Messages

(4 rows affected)

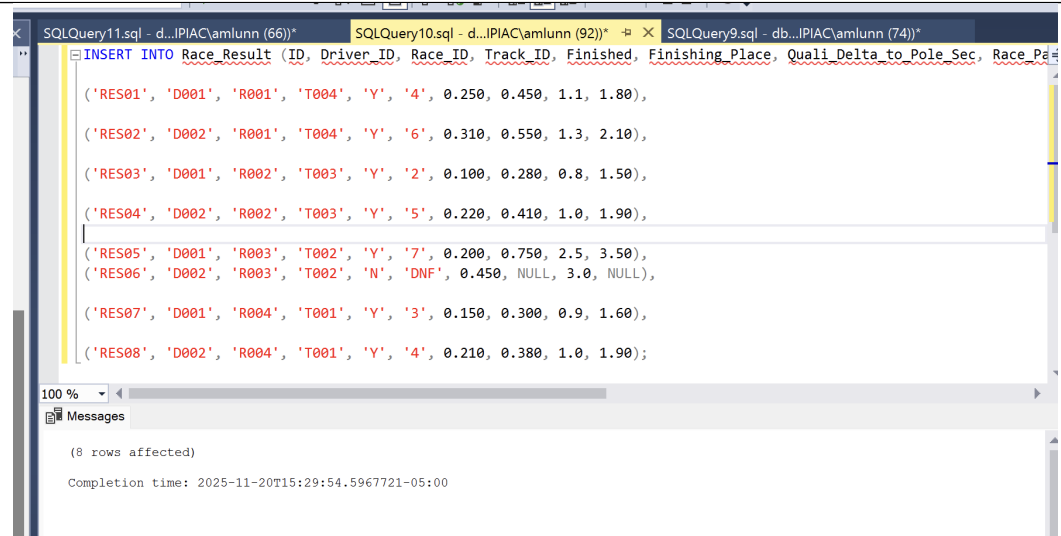
Completion time: 2025-11-20T15:14:24.4493410-05:00

Race Result

```
INSERT INTO Race_Result (ID, Driver_ID, Race_ID, Track_ID, Finished,
Finishing_Place, Quali_Delta_to_Pole_Sec, Race_Pace_Delta_to_Leader_Sec,
Ride_Height_Dev_mm, Tire_Deg_Rate_Percent) VALUES

('RES01', 'D001', 'R001', 'T004', 'Y', '4', 0.250, 0.450, 1.1, 1.80),
('RES02', 'D002', 'R001', 'T004', 'Y', '6', 0.310, 0.550, 1.3, 2.10),
('RES03', 'D001', 'R002', 'T003', 'Y', '2', 0.100, 0.280, 0.8, 1.50),
('RES04', 'D002', 'R002', 'T003', 'Y', '5', 0.220, 0.410, 1.0, 1.90),
('RES05', 'D001', 'R003', 'T002', 'Y', '7', 0.200, 0.750, 2.5, 3.50),
('RES06', 'D002', 'R003', 'T002', 'N', NULL, 0.450, 3.0, NULL, NULL),
('RES07', 'D001', 'R004', 'T001', 'Y', '3', 0.150, 0.300, 0.9, 1.60),
('RES08', 'D002', 'R004', 'T001', 'Y', '4', 0.210, 0.380, 1.0, 1.90)
('RES09','D009','R009','T009','Y',13,0.550,1.100,2.1,4.20),
```

```
(
'RES10','D010','R010','T010','Y',15,0.650,1.300,2.3,4.60),
'RES11','D011','R011','T011','Y',8,0.300,0.650,1.0,2.40),
'RES12','D012','R012','T012','Y',16,0.700,1.400,2.6,4.80),
'RES13','D013','R013','T013','Y',10,0.320,0.780,1.3,2.80),
'RES14','D014','R014','T014','Y',9,0.280,0.700,1.1,2.50),
'RES15','D015','R015','T015','Y',13,0.500,1.000,1.8,3.60),
'RES16','D016','R016','T016','Y',11,0.350,0.850,1.5,3.10),
'RES17','D017','R017','T017','Y',12,0.450,0.950,1.7,3.30),
'RES18','D018','R018','T018','Y',14,0.550,1.200,2.0,3.90),
'RES19','D019','R019','T019','Y',15,0.600,1.300,2.1,4.10),
'RES20','D020','R020','T020','Y',18,0.800,1.700,2.8,5.20);
```



```
SQLQuery11.sql - d:\IPIAC\amlunn (66)*
SQLQuery10.sql - d:\IPIAC\amlunn (92)*
SQLQuery9.sql - db...\IPIAC\amlunn (74)*

INSERT INTO Race Result (ID, Driver_ID, Race_ID, Track_ID, Finished, Finishing_Place, Quali_Delta_to_Pole_Sec, Race_Pd

(
'RES01', 'D001', 'R001', 'T004', 'Y', '4', 0.250, 0.450, 1.1, 1.80),
'RES02', 'D002', 'R001', 'T004', 'Y', '6', 0.310, 0.550, 1.3, 2.10),
'RES03', 'D001', 'R002', 'T003', 'Y', '2', 0.100, 0.280, 0.8, 1.50),
'RES04', 'D002', 'R002', 'T003', 'Y', '5', 0.220, 0.410, 1.0, 1.90),
'RES05', 'D001', 'R003', 'T002', 'Y', '7', 0.200, 0.750, 2.5, 3.50),
'RES06', 'D002', 'R003', 'T002', 'N', 'DNF', 0.450, NULL, 3.0, NULL),
'RES07', 'D001', 'R004', 'T001', 'Y', '3', 0.150, 0.300, 0.9, 1.60),
'RES08', 'D002', 'R004', 'T001', 'Y', '4', 0.210, 0.380, 1.0, 1.90);

100 %
Messages
(8 rows affected)
Completion time: 2025-11-20T15:29:54.5967721-05:00
```

Advanced SQL:

Query 1:

SELECT

E.Last_Name,

CAST(AVG(CASE WHEN T.Smoothness_Index <= 2 THEN
RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) AS DECIMAL(4,3)) AS
Avg_Delta_Bumpy_Tracks,

CAST(AVG(CASE WHEN T.Smoothness_Index >= 4 THEN
RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) AS DECIMAL(4,3)) AS
Avg_Delta_Smooth_Tracks,

-- Calculate the difference (Bumpy - Smooth)

CAST(AVG(CASE WHEN T.Smoothness_Index <= 2 THEN
RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) -

AVG(CASE WHEN T.Smoothness_Index >= 4 THEN RR.Race_Pace_Delta_to_Leader_Sec
ELSE NULL END) AS DECIMAL(4,3)) AS Bumpy_Penalty_Sec

FROM

Race_Result RR

JOIN Driver E ON RR.Driver_ID = E.ID

JOIN Track T ON RR.Track_ID = T.ID

WHERE E.Role = 'Driver' AND RR.Finished = 'Y'

GROUP BY E.Last_Name

ORDER BY Bumpy_Penalty_Sec DESC;

The screenshot shows a SQL query execution window with the following content:

```
SQLQuery2.sql - db...PIAC\drparent (88)* X
SELECT
E.Last_Name,
CAST(AVG(CASE WHEN T.Smoothness_Index <= 2 THEN RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) AS DECIMAL(4,3)) AS Avg_Delta_Bumpy_Tracks,
CAST(AVG(CASE WHEN T.Smoothness_Index >= 4 THEN RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) AS DECIMAL(4,3)) AS Avg_Delta_Smooth_Tracks,
-- Calculate the difference (Bumpy - Smooth)
CAST(AVG(CASE WHEN T.Smoothness_Index <= 2 THEN RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) -
AVG(CASE WHEN T.Smoothness_Index >= 4 THEN RR.Race_Pace_Delta_to_Leader_Sec ELSE NULL END) AS DECIMAL(4,3)) AS Bumpy_Penalty_Sec
FROM
Race_Result RR
JOIN Driver E ON RR.Driver_ID = E.ID
JOIN Track T ON RR.Track_ID = T.ID
WHERE E.Role = 'Driver' AND RR.Finished = 'Y'
GROUP BY E.Last_Name
ORDER BY Bumpy_Penalty_Sec DESC;
```

The results pane shows the following data:

	Last_Name	Avg_Delta_Bumpy_Tracks	Avg_Delta_Smooth_Tracks	Bumpy_Penalty_Sec
1	Lederc	0.800	0.400	0.400
2	Hamilton	NULL	0.500	NULL

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (88) CIS351-01-DRP 00:00:00 2 rows

Query 2:

WITH High_Sensitivity_Races AS (

-- Select races where both ride height deviation and tire degradation were above a performance threshold (e.g., > 1.5mm dev and > 2.0% deg)

SELECT

RR.Driver_ID,

RR.Race_ID,

RR.Ride_Height_Dev_mm,

RR.Tire_Deg_Rate_Percent,

(RR.Ride_Height_Dev_mm * RR.Tire_Deg_Rate_Percent) AS Sensitivity_Metric -- Custom metric to quantify severity

FROM

Race_Result RR

WHERE

RR.Ride_Height_Dev_mm > 1.5

AND RR.Tire_Deg_Rate_Percent > 2.00

AND RR.Finished = 'Y'

)

SELECT

R.Name AS Race_Name,

E.Last_Name,

HSR.Ride_Height_Dev_mm,

HSR.Tire_Deg_Rate_Percent,

HSR.Sensitivity_Metric,

-- Rank the results to prioritize engineering investigation

RANK() OVER (ORDER BY HSR.Sensitivity_Metric DESC) AS Sensitivity_Rank

FROM High_Sensitivity_Races HSR

JOIN Race R ON HSR.Race_ID = R.ID

JOIN Driver E ON HSR.Driver_ID = E.ID

ORDER BY

Sensitivity_Rank;

SQLQuery2.sql - db...PIAC\drparent (88)*

```
WITH High_Sensitivity_Races AS (  
    -- Select races where both ride height deviation and tire degradation were above a performance threshold (e.g., > 1.5mm dev and > 2.0% deg)  
    SELECT  
        RR.Driver_ID,  
        RR.Race_ID,  
        RR.Ride_Height_Dev_mm,  
        RR.Tire_Deg_Rate_Percent,  
        (RR.Ride_Height_Dev_mm * RR.Tire_Deg_Rate_Percent) AS Sensitivity_Metric -- Custom metric to quantify severity  
    FROM  
        Race_Result RR  
    WHERE  
        RR.Ride_Height_Dev_mm > 1.5  
        AND RR.Tire_Deg_Rate_Percent > 2.00  
        AND RR.Finished = 'Y'  
    )  
    SELECT
```

100 %

	Race_Name	Last_Name	Ride_Height_Dev_mm	Tire_Deg_Rate_Percent	Sensitivity_Metric	Sensitivity_Rank
1	Belgian Grand Prix	Fluxa	2.8	5.20	14.560	1
2	Austrian Grand Prix	Iloft	2.6	4.80	12.480	2
3	Dutch Grand Prix	Baldisserri	2.3	4.60	10.580	3
4	Emilia Romagna Grand Prix	Clear	2.1	4.20	8.820	4
5	United States Grand Prix	Lederc	2.5	3.50	8.750	5
6	Las Vegas Grand Prix	Smith	2.1	4.10	8.610	6
7	Mexico City Grand Prix	Montoya	2.0	3.90	7.800	7
8	Chinese Grand Prix	Armstrong	1.8	3.60	6.480	8
9	Canadian Grand Prix	Crawford	1.7	3.30	5.610	9

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (88) CIS351-01-DRP 00:00:00 9 rows

Query 3:

SELECT

E.Last_Name, E.ID,

-- Calculate the average performance gap

CAST(AVG(RR.Quali_Delta_to_Pole_Sec) AS DECIMAL(4,3)) AS Avg_Quali_Delta,

-- Calculate the variability (consistency) of the gap

CAST(STDEV(RR.Quali_Delta_to_Pole_Sec) OVER (PARTITION BY E.ID) AS

DECIMAL(4,3)) AS Quali_Consistency_STDEV

FROM

Race_Result RR

JOIN Driver E ON RR.Driver_ID = E.ID

WHERE E.Role = 'Driver'

GROUP BY E.ID, Last_Name, Quali_Delta_to_Pole_Sec

ORDER BY Quali_Consistency_STDEV ASC; -- Lower STDEV means more consistent driver

The screenshot shows the SQL Developer interface. The top pane displays the following SQL query:

```
SELECT
  E.Last_Name, E.ID,
  -- Calculate the average performance gap
  CAST(AVG(RR.Quali_Delta_to_Pole_Sec) AS DECIMAL(4,3)) AS Avg_Quali_Delta,
  -- Calculate the variability (consistency) of the gap
  CAST(STDEV(RR.Quali_Delta_to_Pole_Sec) OVER (PARTITION BY E.ID) AS DECIMAL(4,3)) AS Quali_Consistency_STDEV
FROM
  Race_Result RR
JOIN Driver E ON RR.Driver_ID = E.ID
WHERE E.Role = 'Driver'
GROUP BY E.ID, Last_Name, Quali_Delta_to_Pole_Sec
ORDER BY Quali_Consistency_STDEV ASC; -- Lower STDEV means more consistent driver
```

The bottom pane shows the query results in a table with 8 rows. The columns are Last_Name, ID, Avg_Quali_Delta, and Quali_Consistency_STDEV.

	Last_Name	ID	Avg_Quali_Delta	Quali_Consistency_STDEV
1	Lederc	D001	0.100	0.065
2	Lederc	D001	0.150	0.065
3	Lederc	D001	0.200	0.065
4	Lederc	D001	0.250	0.065
5	Hamilton	D002	0.210	0.111
6	Hamilton	D002	0.220	0.111
7	Hamilton	D002	0.310	0.111
8	Hamilton	D002	0.450	0.111

The status bar at the bottom indicates: Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (88) CIS351-01-DRP 00:00:00 8 rows

Query 4:

SELECT

T.Name AS Track_Name,

R.Name AS Race_Name,

SUM(CASE

-- Award points for low Race Pace Delta (good race pace)

WHEN RR.Race_Pace_Delta_to_Leader_Sec IS NOT NULL AND
RR.Race_Pace_Delta_to_Leader_Sec < 0.400 THEN 10

-- Deduct points for high Ride Height Deviation (narrow window missed)

WHEN RR.Ride_Height_Dev_mm > 2.0 THEN -8

-- Deduct points for high Tire Degradation (car destroying tires)

WHEN RR.Tire_Deg_Rate_Percent > 2.50 THEN -5

-- Award points for a strong finishing position (top 5)

WHEN RR.Finishing_Place IS NOT NULL AND CAST(RR.Finishing_Place AS INT) <= 5
THEN 7

ELSE 0

END) AS Engineering_Review_Score

FROM Race_Result RR

JOIN Race R ON RR.Race_ID = R.ID

JOIN Track T ON R.Track_ID = T.ID

WHERE RR.Finished = 'Y' -- Only analyze completed races

GROUP BY T.Name, R.Name

ORDER BY Engineering_Review_Score DESC;

SQLQuery2.sql - db...PIAC\drparent (88)*

```

SELECT
T.Name AS Track_Name,
R.Name AS Race_Name,
SUM(CASE
-- Award points for low Race Pace Delta (good race pace)
WHEN RR.Race_Pace_Delta_to_Leader_Sec IS NOT NULL AND RR.Race_Pace_Delta_to_Leader_Sec < 0.400 THEN 10
-- Deduct points for high Ride Height Deviation (narrow window missed)
WHEN RR.Ride_Height_Dev_mm > 2.0 THEN -8
-- Deduct points for high Tire Degradation (car destroying tires)
) AS Engineering_Review_Score
FROM Track T
JOIN Race R ON T.Name = R.Track_Name
JOIN Race_Result RR ON R.Race_ID = RR.Race_ID

```

100 %

Results Messages

	Track_Name	Race_Name	Engineering_Review_Score
1	Monza	Italian Grand Prix	17
2	Marina Bay Street Circuit	Singapore Grand Prix	17
3	Silverstone Circuit	British Grand Prix	7
4	Hungaroring	Hungarian Grand Prix	0
5	Losail International Circuit	Qatar Grand Prix	0
6	Jeddah Corniche Circuit	Saudi Arabian Grand Prix	-5
7	Mexico City Autodromo Hermanos Rodriguez	Mexico City Grand Prix	-5
8	Montreal Circuit Gilles Villeneuve	Canadian Grand Prix	-5
9	Shanghai International Circuit	Chinese Grand Prix	-5
10	Albert Park	Australian Grand Prix	-5
11	Red Bull Ring	Austrian Grand Prix	-8
12	Spa-Francorchamps	Belgian Grand Prix	-8
13	Las Vegas Strip Circuit	Las Vegas Grand Prix	-8
14	Zandvoort	Dutch Grand Prix	-8
15	Imola	Emilia Romagna Grand Prix	-8
16	Circuit of the Americas (COTA)	United States Grand Prix	-8

Query executed successfully.

db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (88) CIS351-01-DRP 00:00:00 16 rows

AI-Ready Feature View:

- Feature Engineering View for Ferrari SF-25 Performance Optimization
- Prepares structured data for ML models to identify performance inconsistencies

CREATE VIEW vw_ferrari_performance_features AS
WITH

-- Base performance data with track characteristics

base_performance AS (

SELECT

rr.ID as result_id,

rr.Race_ID,

r.Date as race_date,

r.Name as race_name,

rr.Driver_ID,

CONCAT(d.First_Name, ' ', d.Last_Name) as driver_name,

d.Role as driver_role,

DATEDIFF(day, d.Date_Hired, r.Date) as driver_experience_days,

rr.Track_ID,

t.Name as track_name,

t.Location as track_location,

t.Smoothness_Index,

t.Capacity as track_capacity,

rr.Finished as finished_race,

rr.Finishing_Place,

rr.Quali_Delta_to_Pole_Sec,

rr.Race_Pace_Delta_to_Leader_Sec,

rr.Tire_Deg_Rate_Percent

```

FROM Race_Result rr
JOIN Race r ON rr.Race_ID = r.ID
JOIN Track t ON rr.Track_ID = t.ID
JOIN Driver d ON rr.Driver_ID = d.ID
),

-- Track type classification based on smoothness
track_classification AS (
SELECT
    Track_ID,
    track_name,
    Smoothness_Index,
    CASE
        WHEN Smoothness_Index >= 80 THEN 'High_Speed_Smooth'
        WHEN Smoothness_Index >= 60 THEN 'Medium_Speed_Mixed'
        WHEN Smoothness_Index >= 40 THEN 'Technical_Bumpy'
        ELSE 'Street_Circuit'
    END as track_type,
    CASE
        WHEN track_capacity > 100000 THEN 'Major'
        WHEN track_capacity > 50000 THEN 'Standard'
        ELSE 'Compact'
    END as track_size_category
FROM (SELECT DISTINCT Track_ID, track_name, Smoothness_Index, track_capacity
FROM base_performance) t
),

-- Driver rolling performance metrics (last 5 races)
driver_recent_form AS (
SELECT
    result_id,
    Driver_ID,
    race_date,
    AVG(Finishing_Place) OVER (
        PARTITION BY Driver_ID
        ORDER BY race_date
        ROWS BETWEEN 5 PRECEDING AND 1 PRECEDING
    ) as avg_finish_last_5_races,
    AVG(Tire_Deg_Rate_Percent) OVER (
        PARTITION BY Driver_ID
        ORDER BY race_date
        ROWS BETWEEN 5 PRECEDING AND 1 PRECEDING
    ) as avg_tire_deg_last_5_races,
    COUNT(CASE WHEN finished_race = 'N' THEN 1 END) OVER (
        PARTITION BY Driver_ID
        ORDER BY race_date

```

```

        ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
    ) as dnf_count_last_10_races
FROM base_performance
),

-- Qualifying vs Race consistency metrics
consistency_metrics AS (
    SELECT
        result_id,
        Driver_ID,
        -- Quali-to-race delta (negative means lost positions)
        (Quali_Delta_to_Pole_Sec - Race_Pace_Delta_to_Leader_Sec) as
quali_race_pace_variance,
        -- Historical consistency per driver
        STDEV(Quali_Delta_to_Pole_Sec) OVER (
            PARTITION BY Driver_ID
            ORDER BY race_date
            ROWS BETWEEN 10 PRECEDING AND CURRENT ROW
        ) as quali_consistency_stddev,
        STDEV(Race_Pace_Delta_to_Leader_Sec) OVER (
            PARTITION BY Driver_ID
            ORDER BY race_date
            ROWS BETWEEN 10 PRECEDING AND CURRENT ROW
        ) as race_pace_consistency_stddev
    FROM base_performance
),

-- Track-specific historical performance
track_history AS (
    SELECT
        result_id,
        Track_ID,
        Driver_ID,
        AVG(Finishing_Place) OVER (
            PARTITION BY Track_ID, Driver_ID
            ORDER BY race_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
        ) as historical_avg_finish_at_track,
        AVG(Tire_Deg_Rate_Percent) OVER (
            PARTITION BY Track_ID, Driver_ID
            ORDER BY race_date
            ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
        ) as historical_tire_deg_at_track,
        COUNT(*) OVER (
            PARTITION BY Track_ID, Driver_ID
            ORDER BY race_date

```

```

        ROWS BETWEEN UNBOUNDED PRECEDING AND 1 PRECEDING
    ) as races_at_track_count
FROM base_performance
),

-- Track type performance aggregation
track_type_performance AS (
    SELECT
        bp.result_id,
        tc.track_type,
        AVG(bp.Finishing_Place) OVER (
            PARTITION BY tc.track_type
            ORDER BY bp.race_date
            ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
        ) as avg_finish_by_track_type,
        AVG(bp.Tire_Deg_Rate_Percent) OVER (
            PARTITION BY tc.track_type
            ORDER BY bp.race_date
            ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
        ) as avg_tire_deg_by_track_type,
        COUNT(CASE WHEN bp.finished_race = 'N' THEN 1 END) OVER (
            PARTITION BY tc.track_type
            ORDER BY bp.race_date
            ROWS BETWEEN 10 PRECEDING AND 1 PRECEDING
        ) as dnf_count_by_track_type
    FROM base_performance bp
    JOIN track_classification tc ON bp.Track_ID = tc.Track_ID
),

-- Setup sensitivity indicators (anomaly detection features)
setup_sensitivity AS (
    SELECT
        result_id,
        -- Large swings in tire deg suggest setup sensitivity
        ABS(Tire_Deg_Rate_Percent - AVG(Tire_Deg_Rate_Percent) OVER (
            ORDER BY race_date
            ROWS BETWEEN 5 PRECEDING AND 1 PRECEDING
        )) as tire_deg_deviation_from_avg,
        -- Large quali-race performance gaps suggest instability
        ABS(Quali_Delta_to_Pole_Sec - Race_Pace_Delta_to_Leader_Sec) as
quali_race_performance_gap,
        -- Position volatility
        STDEV(Finishing_Place) OVER (
            ORDER BY race_date
            ROWS BETWEEN 5 PRECEDING AND CURRENT ROW
        ) as finishing_position_volatility

```

```

    FROM base_performance
)

-- Final feature set assembly
SELECT
    -- Identifiers
    bp.result_id,
    bp.Race_ID,
    bp.race_date,
    bp.race_name,
    bp.Driver_ID,
    bp.driver_name,
    bp.driver_role,
    bp.Track_ID,
    bp.track_name,
    bp.track_location,

    -- Target variables
    bp.Finishing_Place as target_finish_position,
    bp.finished_race as target_finished,

    -- Track characteristics
    bp.Smoothness_Index,
    bp.track_capacity,
    tc.track_type,
    tc.track_size_category,

    -- Driver features
    bp.driver_experience_days,
    drf.avg_finish_last_5_races,
    drf.avg_tire_deg_last_5_races,
    drf.dnf_count_last_10_races,

    -- Performance metrics
    bp.Quali_Delta_to_Pole_Sec,
    bp.Race_Pace_Delta_to_Leader_Sec,
    bp.Tire_Deg_Rate_Percent,

    -- Consistency indicators (key for identifying instability)
    cm.quali_race_pace_variance,
    cm.quali_consistency_stddev,
    cm.race_pace_consistency_stddev,

    -- Track-specific history
    COALESCE(th.historical_avg_finish_at_track, 10.0) as historical_avg_finish_at_track,

```



```

    COALESCE(th.historical_tire_deg_at_track, bp.Tire_Deg_Rate_Percent) as
historical_tire_deg_at_track,
    COALESCE(th.races_at_track_count, 0) as races_at_track_count,

-- Track type performance patterns
    COALESCE(ttp.avg_finish_by_track_type, 10.0) as avg_finish_by_track_type,
    COALESCE(ttp.avg_tire_deg_by_track_type, bp.Tire_Deg_Rate_Percent) as
avg_tire_deg_by_track_type,
    COALESCE(ttp.dnf_count_by_track_type, 0) as dnf_count_by_track_type,

-- Setup sensitivity features (critical for Ferrari's problem)
    ss.tire_deg_deviation_from_avg,
    ss.quali_race_performance_gap,
    ss.finishing_position_volatility,

-- Derived flags for defect identification
CASE
    WHEN bp.finished_race = 'N' THEN 1
    ELSE 0
END as flag_dnf,
CASE
    WHEN ss.quali_race_performance_gap > 2.0 THEN 1
    ELSE 0
END as flag_high_quali_race_variance,
CASE
    WHEN ss.tire_deg_deviation_from_avg > 15.0 THEN 1
    ELSE 0
END as flag_abnormal_tire_degradation,
CASE
    WHEN ss.finishing_position_volatility > 5.0 THEN 1
    ELSE 0
END as flag_high_position_volatility,

-- Operating window indicator (combined instability score)
(COALESCE(ss.quali_race_performance_gap, 0) * 0.3 +
 COALESCE(ss.tire_deg_deviation_from_avg, 0) * 0.3 +
 COALESCE(ss.finishing_position_volatility, 0) * 2.0 +
 COALESCE(cm.race_pace_consistency_stddev, 0) * 10.0) as
narrow_operating_window_score

FROM base_performance bp
LEFT JOIN track_classification tc ON bp.Track_ID = tc.Track_ID
LEFT JOIN driver_recent_form drf ON bp.result_id = drf.result_id
LEFT JOIN consistency_metrics cm ON bp.result_id = cm.result_id
LEFT JOIN track_history th ON bp.result_id = th.result_id
LEFT JOIN track_type_performance ttp ON bp.result_id = ttp.result_id

```

LEFT JOIN setup_sensitivity ss ON bp.result_id = ss.result_id

SQLQuery2.sql - db...PIAC\drparent (88))

```
-- Feature Engineering View for Ferrari SF-25 Performance Optimization
-- Prepares structured data for ML models to identify performance inconsistencies

CREATE VIEW vw_ferrari_performance_features3 AS
WITH
-- Base performance data with track characteristics
base_performance AS (
    SELECT
        rr.ID as result_id,
        rr.Race_ID,
        r.Date as race_date,
        r.Name as race_name,
        rr.Driver_ID,
        CONCAT(d.First_Name, ' ', d.Last_Name) as driver_name,
        d.Role as driver_role,
        DATEDIFF(day, d.Date_Hired, r.Date) as driver_experience_days,
        rr.Track_ID,
        t.Name as track_name,
        t.Location as track_location,
        t.Smoothness_Index,
        t.Capacity as track_capacity
    FROM
        Results r
        JOIN Races rr ON r.Race_ID = rr.Race_ID
        JOIN Drivers d ON rr.Driver_ID = d.Driver_ID
        JOIN Tracks t ON rr.Track_ID = t.Track_ID
)

SELECT * FROM base_performance
```

100 %

Messages

Commands completed successfully.

Completion time: 2025-11-21T17:30:53.1569370-05:00

100 %

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (88) CIS351-01-DRP 00:00:00 0 rows

SQLQuery3.sql - db...PIAC\drparent (63))

```
SELECT *
FROM vw_ferrari_performance_features3
```

100 %

Results Messages

	result_id	Race_ID	race_date	race_name	Driver_ID	driver_name	driver_role	Track_ID	track_name	track_location	target_finish_position	target_finished
1	RES01	R001	2025-07-06	British Grand Prix	D001	Charles Leclerc	Driver	T004	Silverstone Circuit	Silverstone, UK	4	Y
2	RES02	R001	2025-07-06	British Grand Prix	D002	Lewis Hamilton	Driver	T004	Silverstone Circuit	Silverstone, UK	6	Y
3	RES03	R002	2025-09-21	Singapore Grand Prix	D001	Charles Leclerc	Driver	T003	Marina Bay Street Circuit	Singapore	2	Y
4	RES04	R002	2025-09-21	Singapore Grand Prix	D002	Lewis Hamilton	Driver	T003	Marina Bay Street Circuit	Singapore	5	Y
5	RES05	R003	2025-10-19	United States Grand Prix	D001	Charles Leclerc	Driver	T002	Circuit of the Americas (COTA)	Austin, USA	7	Y
6	RES06	R003	2025-10-19	United States Grand Prix	D002	Lewis Hamilton	Driver	T002	Circuit of the Americas (COTA)	Austin, USA	NULL	N
7	RES07	R004	2025-09-07	Italian Grand Prix	D001	Charles Leclerc	Driver	T001	Monza	Monza, Italy	3	Y
8	RES08	R004	2025-09-07	Italian Grand Prix	D002	Lewis Hamilton	Driver	T001	Monza	Monza, Italy	4	Y
9	RES09	R009	2025-05-18	Emilia Romagna Grand Prix	D009	Jock Clear	Simulator Driver	T009	Imola	Imola, Italy	13	Y
10	RES10	R010	2025-08-24	Dutch Grand Prix	D010	Luca Baldisserri	Simulator Driver	T010	Zandvoort	Netherlands	15	Y
11	RES11	R011	2025-07-20	Hungarian Grand Prix	D011	Giuliano Alesi	Academy Driver	T011	Hungaroring	Budapest, Hungary	8	Y
12	RES12	R012	2025-06-29	Austrian Grand Prix	D012	Callum Ilott	Test Driver	T012	Red Bull Ring	Spielberg, Austria	16	Y
13	RES13	R013	2025-03-23	Saudi Arabian Grand Prix	D013	Mick Schumacher	Reserve Driver	T013	Jeddah Corniche Circuit	Jeddah, Saudi Arabia	10	Y
14	RES14	R014	2025-11-30	Qatar Grand Prix	D014	Antonio Giovinazzi	Test Driver	T014	Losail International Circuit	Qatar	9	Y
15	RES15	R015	2025-04-27	Chinese Grand Prix	D015	Marcus Armstrong	Academy Driver	T015	Shanghai International Circuit	Shanghai, China	13	Y
16	RES16	R016	2025-03-16	Australian Grand Prix	D016	Enzo Fittipaldi	Academy Driver	T016	Albert Park	Melbourne, Australia	11	Y
17	RES17	R017	2025-06-15	Canadian Grand Prix	D017	Jak Crawford	Academy Driver	T017	Montreal Circuit Gilles Villeneuve	Montreal, Canada	12	Y
18	RES18	R018	2025-10-26	Mexico City Grand Prix	D018	Sebastian Montoya	Academy Driver	T018	Mexico City Autodromo Hermanos Rodriguez	Mexico City, Mexico	14	Y
19	RES19	R019	2025-11-22	Las Vegas Grand Prix	D019	Bear Smith	Simulator Driver	T019	Las Vegas Strip Circuit	Las Vegas, USA	15	Y
20	RES20	R020	2025-08-03	Belgian Grand Prix	D020	Lorenzo Fluxa	Academy Driver	T020	Spa-Francorchamps	Spa, Belgium	18	Y

100 %

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (63) CIS351-01-DRP 00:00:00 20 rows

-- Stored Procedure: Comprehensive Ferrari SF-25 Performance Analysis
-- Identifies root causes of instability and inconsistency across track types

```
CREATE PROCEDURE sp_analyze_ferrari_performance
    @DriverID INT = NULL,          -- Optional: specific driver, NULL = all drivers
    @StartDate DATE = NULL,        -- Optional: analysis start date
    @EndDate DATE = NULL,          -- Optional: analysis end date
    @MinInstabilityScore FLOAT = 20.0 -- Threshold for flagging high instability
AS
BEGIN
    SET NOCOUNT ON;

    -- Set default date range if not provided (last 12 months)
    IF @StartDate IS NULL
        SET @StartDate = DATEADD(MONTH, -12, GETDATE());

    IF @EndDate IS NULL
        SET @EndDate = GETDATE();

    -- =====
    -- RESULT SET 1: Executive Summary
    -- =====
    SELECT
        'EXECUTIVE SUMMARY' as report_section,
        COUNT(DISTINCT race_date) as total_races_analyzed,
        COUNT(DISTINCT Driver_ID) as drivers_analyzed,
        AVG(target_finish_position) as avg_finishing_position,
        AVG(narrow_operating_window_score) as avg_instability_score,
        SUM(flag_dnf) as total_dnfs,
        SUM(flag_high_quali_race_variance) as races_with_quali_race_inconsistency,
        SUM(flag_abnormal_tire_degradation) as races_with_tire_issues,
        CAST(SUM(flag_dnf) * 100.0 / COUNT(*) AS DECIMAL(5,2)) as dnf_percentage
    FROM vw_ferrari_performance_features
    WHERE race_date BETWEEN @StartDate AND @EndDate
        AND (@DriverID IS NULL OR Driver_ID = @DriverID);

    -- =====
    -- RESULT SET 2: Track Type Performance Analysis
    -- Identifies which circuit types cause the most problems
    -- =====
    SELECT
        'TRACK TYPE ANALYSIS' as report_section,
        track_type,
        COUNT(*) as races_count,
```

```

    AVG(target_finish_position) as avg_finish,
    AVG(narrow_operating_window_score) as avg_instability,
    AVG(quali_race_performance_gap) as avg_quali_race_gap,
    AVG(Tire_Deg_Rate_Percent) as avg_tire_degradation,
    SUM(flag_dnf) as dnf_count,
    -- Identify problem severity
    CASE
        WHEN AVG(narrow_operating_window_score) > 30 THEN 'CRITICAL - Major
Setup Issues'
        WHEN AVG(narrow_operating_window_score) > 20 THEN 'HIGH - Significant
Instability'
        WHEN AVG(narrow_operating_window_score) > 10 THEN 'MODERATE - Some
Inconsistency'
        ELSE 'LOW - Stable Performance'
    END as problem_severity
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
GROUP BY track_type
ORDER BY avg_instability DESC;

```

```

-- =====
-- RESULT SET 3: Specific Problematic Races
-- Lists individual races with high instability scores
-- =====

```

```

SELECT
    'PROBLEMATIC RACES' as report_section,
    race_date,
    race_name,
    track_name,
    track_type,
    driver_name,
    target_finish_position,
    narrow_operating_window_score,
    quali_race_performance_gap,
    tire_deg_deviation_from_avg,
    finishing_position_volatility,
    -- Diagnosis
    CASE
        WHEN flag_high_quali_race_variance = 1 AND flag_abnormal_tire_degradation = 1
            THEN 'Setup Sensitivity + Tire Management Issues'
        WHEN flag_high_quali_race_variance = 1
            THEN 'Major Quali-to-Race Performance Drop'
        WHEN flag_abnormal_tire_degradation = 1
            THEN 'Abnormal Tire Degradation Pattern'
    END

```

```

        WHEN flag_high_position_volatility = 1
        THEN 'Inconsistent Race Pace'
        ELSE 'General Instability'
    END as issue_diagnosis
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
    AND narrow_operating_window_score > @MinInstabilityScore
ORDER BY narrow_operating_window_score DESC;

-- =====
-- RESULT SET 4: Track Smoothness Correlation
-- Determines if track surface quality impacts performance
-- =====
SELECT
    'TRACK SMOOTHNESS IMPACT' as report_section,
    CASE
        WHEN Smoothness_Index >= 80 THEN '80-100: Very Smooth'
        WHEN Smoothness_Index >= 60 THEN '60-79: Smooth'
        WHEN Smoothness_Index >= 40 THEN '40-59: Moderate'
        ELSE '0-39: Bumpy'
    END as smoothness_category,
    COUNT(*) as race_count,
    AVG(target_finish_position) as avg_finish,
    AVG(narrow_operating_window_score) as avg_instability,
    AVG(Tire_Deg_Rate_Percent) as avg_tire_deg,
    STDEV(target_finish_position) as finish_position_stddev
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
GROUP BY
    CASE
        WHEN Smoothness_Index >= 80 THEN '80-100: Very Smooth'
        WHEN Smoothness_Index >= 60 THEN '60-79: Smooth'
        WHEN Smoothness_Index >= 40 THEN '40-59: Moderate'
        ELSE '0-39: Bumpy'
    END
ORDER BY smoothness_category;

-- =====
-- RESULT SET 5: Driver Comparison
-- Identifies if issues are car-related or driver-related
-- =====
SELECT

```

```

'DRIVER PERFORMANCE COMPARISON' as report_section,
driver_name,
COUNT(*) as races,
AVG(target_finish_position) as avg_finish,
AVG(narrow_operating_window_score) as avg_instability,
AVG(quali_race_performance_gap) as avg_quali_race_gap,
SUM(flag_dnf) as dnfs,
AVG(Tire_Deg_Rate_Percent) as avg_tire_deg,
-- Consistency rating
CASE
    WHEN STDEV(target_finish_position) < 3 THEN 'Very Consistent'
    WHEN STDEV(target_finish_position) < 5 THEN 'Consistent'
    WHEN STDEV(target_finish_position) < 7 THEN 'Inconsistent'
    ELSE 'Very Inconsistent'
END as consistency_rating
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
GROUP BY driver_name
ORDER BY avg_instability DESC;

-- =====
-- RESULT SET 6: Recommendations
-- Data-driven recommendations for improvement
-- =====
WITH recommendations AS (
    SELECT
        track_type,
        AVG(narrow_operating_window_score) as avg_score,
        COUNT(*) as race_count
    FROM vw_ferrari_performance_features
    WHERE race_date BETWEEN @StartDate AND @EndDate
        AND (@DriverID IS NULL OR Driver_ID = @DriverID)
    GROUP BY track_type
)
SELECT
    'RECOMMENDATIONS' as report_section,
    1 as priority,
    'Focus setup development on ' + track_type + ' circuits' as recommendation,
    'Avg Instability: ' + CAST(ROUND(avg_score, 2) AS VARCHAR) as supporting_data
FROM recommendations
WHERE avg_score = (SELECT MAX(avg_score) FROM recommendations)

UNION ALL

```

```

SELECT
    'RECOMMENDATIONS',
    2,
    'Investigate tire degradation patterns - ' +
    CAST(COUNT(*) AS VARCHAR) + ' races with abnormal tire wear',
    'Review tire pressure and temperature management'
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
    AND flag_abnormal_tire_degradation = 1
HAVING COUNT(*) > 0

UNION ALL

SELECT
    'RECOMMENDATIONS',
    3,
    'Address quali-to-race performance drop - ' +
    CAST(COUNT(*) AS VARCHAR) + ' races with major variance',
    'Setup may be optimized for qualifying at expense of race pace'
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
    AND flag_high_quali_race_variance = 1
HAVING COUNT(*) > 0

UNION ALL

SELECT
    'RECOMMENDATIONS',
    4,
    'Expand operating window testing',
    'High position volatility suggests narrow setup window'
FROM vw_ferrari_performance_features
WHERE race_date BETWEEN @StartDate AND @EndDate
    AND (@DriverID IS NULL OR Driver_ID = @DriverID)
    AND flag_high_position_volatility = 1
HAVING COUNT(*) > 0

ORDER BY priority;

END;
GO

-- USAGE EXAMPLES
-- Run full analysis for all drivers in last 12 months

```

```
-- EXEC sp_analyze_ferrari_performance;

-- Analyze specific driver
-- EXEC sp_analyze_ferrari_performance @DriverID = 1;

-- Custom date range analysis
-- EXEC sp_analyze_ferrari_performance
--   @StartDate = '2025-01-01',
--   @EndDate = '2025-10-31';

-- Focus on severe instability cases only
-- EXEC sp_analyze_ferrari_performance
--   @MinInstabilityScore = 30.0;
```

SQLQuery3.sql - db...PIAC\drparent (63))*

```
-- Stored Procedure: Comprehensive Ferrari SF-25 Performance Analysis
-- Identifies root causes of instability and inconsistency across track types

CREATE PROCEDURE sp_analyze_ferrari_performance2
    @DriverID INT = NULL,          -- Optional: specific driver, NULL = all drivers
    @StartDate DATE = NULL,        -- Optional: analysis start date
    @EndDate DATE = NULL,         -- Optional: analysis end date
    @MinInstabilityScore FLOAT = 20.0 -- Threshold for flagging high instability
AS
BEGIN
    SET NOCOUNT ON;

    -- Set default date range if not provided (last 12 months)
    IF @StartDate IS NULL
        SET @StartDate = DATEADD(MONTH, -12, GETDATE());

    IF @EndDate IS NULL
        SET @EndDate = GETDATE();

    -- =====
    -- RESULT SET 1: Executive Summary
    -- =====
```

100 %

Messages

Commands completed successfully.

Completion time: 2025-11-21T17:36:34.1902581-05:00

100 %

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (63) CIS351-01-DRP 00:00:00 0 rows

SQLQuery4.sql - db...PIAC\drparent (72))*

EXEC sp_analyze_ferrari_performance

100 %

Results Messages

report_section	total_races_analyzed	drivers_analyzed	avg_finishing_position	avg_instability_score	total_dnfs	races_with_quali_race_inconsistency	races_with_tire_issues	dnf_percentage
EXECUTIVE SUMMARY	14	12	9.470588	8.608537016068	1	0		5.56

report_section	track_type	races_count	avg_finish	avg_instability	avg_quali_race_gap	avg_tire_degradation	dnf_count	problem_severity
TRACK TYPE ANALYSIS	Street_Circuit	18	9.470588	8.608537016068	0.458235	3.070588	1	LOW - Stable Performance

report_section	race_date	race_name	track_name	track_type	driver_name	target_finish_position	narrow_operating_window_score	quali_race_performance_gap	tire_deg_deviation_from_avg	finishing_
----------------	-----------	-----------	------------	------------	-------------	------------------------	-------------------------------	----------------------------	-----------------------------	------------

report_section	smoothness_category	race_count	avg_finish	avg_instability	avg_tire_deg	finish_position_stddev
TRACK SMOOTHNESS IMPACT	0-39: Bumpy	18	9.470588	8.608537016068	3.070588	4.98895839653459

report_section	driver_name	races	avg_finish	avg_instability	avg_quali_race_gap	dnfs	avg_tire_deg	consistency_rating
DRIVER PERFORMANCE COMPARISON	Lederc	4	4.000000	12.6331563540931	0.300000	0	2.100000	Very Consistent
DRIVER PERFORMANCE COMPARISON	Baldisserri	1	15.000000	12.355684733705	0.650000	0	4.600000	Very Inconsistent
DRIVER PERFORMANCE COMPARISON	Fluxa	1	18.000000	12.1850314495801	0.900000	0	5.200000	Very Inconsistent
DRIVER PERFORMANCE COMPARISON	Hamilton	4	5.000000	11.0348330630769	0.220000	1	1.966666	Very Consistent
DRIVER PERFORMANCE COMPARISON	Montoya	1	14.000000	9.93538460737146	0.650000	0	3.900000	Very Inconsistent
DRIVER PERFORMANCE COMPARISON	Alesi	1	8.000000	9.52987457146399	0.400000	0	2.400000	Very Inconsistent
DRIVER PERFORMANCE COMPARISON	Ilott	1	16.000000	4.77728827066554	0.700000	0	4.800000	Very Inconsistent
DRIVER PERFORMANCE COMPARISON	Clear	1	13.000000	3.475	0.550000	0	4.200000	Very Inconsistent

report_section	priority	recommendation	supporting_data
RECOMMENDATIONS	1	Focus setup development on Street_Circuit circuits	Avg Instability: 8.61
RECOMMENDATIONS	4	Expand operating window testing	High position volatility suggests narrow setup w..

100 %

Query executed successfully. db-opal-ugs (16.0 RTM) QUINNIPIAC\drparent (72) CIS351-01-DRP 00:00:00 17 rows

